

EMWAC Project: Water Quality Monitoring System
Technical Report

Author: Nnamdi Ugwuoke

Date: 12/12/2025

Table of Contents

- Executive Summary
- Introduction
- System Overview
- Hardware Architecture
- Component Descriptions
- Sensor Calibration Procedures
- Wiring and Integration (with pin mapping)
- Testing and Deployment
- Conclusion
- Appendix
- Satellite Configuration images and codes.

Executive Summary

The EMWAC Water Quality Monitoring System is a standalone, solar-powered platform for continuous monitoring of environmental conditions in surface and ground water. The prototype combines multiple industrial-grade water quality sensors with a dual-microcontroller architecture and satellite-based IoT communication.

The system is designed to operate in remote or infrastructure-poor locations where conventional telemetry and manual sampling are impractical. It measures pH, dissolved oxygen, electrical conductivity, temperature, flow rate, ammonium concentration, and nitrate concentration. Measurements are processed locally and transmitted via NB-IoT/LTE-M and Lacuna Space satellite services to a cloud platform for visualization, storage, and analysis.

This report documents the hardware architecture, sensor selection, calibration procedures, and integration approach used to build the prototype. It is intended as a reference for engineers and practitioners who wish to replicate, adapt, or extend the system.



Figure 1. EMWAC Device

1. Introduction

Many water bodies located in rural communities, agricultural zones, or environmentally sensitive regions lack continuous, high-quality water quality monitoring. Ad-hoc sampling or manual testing often fails to capture rapid changes in water conditions caused by pollution events, storm runoff, or operational disturbances.

The EMWAC prototype addresses this gap with a compact, rugged monitoring system that can be deployed at fixed sites such as rivers, ponds, reservoirs, irrigation channels, or industrial wastewater outlets. The device supports long-term unattended operation through low-power electronics, solar energy, and robust sensor technology.

Key objectives of the EMWAC design include:

- Support for multiple water quality parameters in a single compact platform.
- Compatibility with both analog and digital industrial sensors.
- Cabled probes capable of long-term immersion in harsh aquatic environments.
- Reliable communication from remote sites using NB-IoT/LTE-M and satellite links.
- Ease of maintenance, including straightforward calibration and sensor replacement.

2. System Overview

The EMWAC prototype is organized around two cooperating microcontroller boards. An Arduino UNO R4 functions as the primary controller responsible for periodic sampling, signal conditioning, and local processing of sensor data. An Arduino MKR 1500 manages wide-area communication using NB-IoT/LTE-M and forwards processed measurements to a cloud endpoint and satellite link,

enabling near-global coverage via Lacuna Space.

The sensing array consists of seven water quality sensors:

- pH sensor (industrial analog probe)
- Dissolved oxygen sensor (galvanic type)
- Electrical conductivity (EC) sensor with temperature compensation
- Temperature probe
- Flow sensor (Hall-effect turbine type)
- Ammonium ion-selective electrode (NH₄⁺ ISE)
- Nitrate ion-selective electrode (NO₃⁻ ISE)

A 12 V battery and DC–DC converters provide regulated 5 V and 3.3 V rails. All electronics are housed in a weatherproof enclosure, while probes are mounted in the water column or in a flow-through cell. A 2.0-inch TFT LCD provides local visualization of key parameters and system status during installation and maintenance.

3. Hardware Architecture

The hardware is divided into modular subsystems so that each part can be developed, tested, and replaced independently:

- Processing subsystem: Arduino UNO R4 configured as the main data acquisition unit; Arduino MKR 1500 performing communication and higher-level messaging.
- Power subsystem: 12 V energy storage, optionally charged from a solar panel, with step-down regulators for 5 V and 3.3 V outputs.
- Sensing subsystem: seven water quality sensors interfaced through Gravity connectors, BNC connectors, and analog input channels.
- User interface subsystem: TFT LCD display for local feedback.
- Mechanical subsystem: sealed enclosure, probe holders, and cable glands for strain relief and ingress protection.

This modular approach simplifies debugging and makes it easier to reuse the architecture in other environmental monitoring applications.

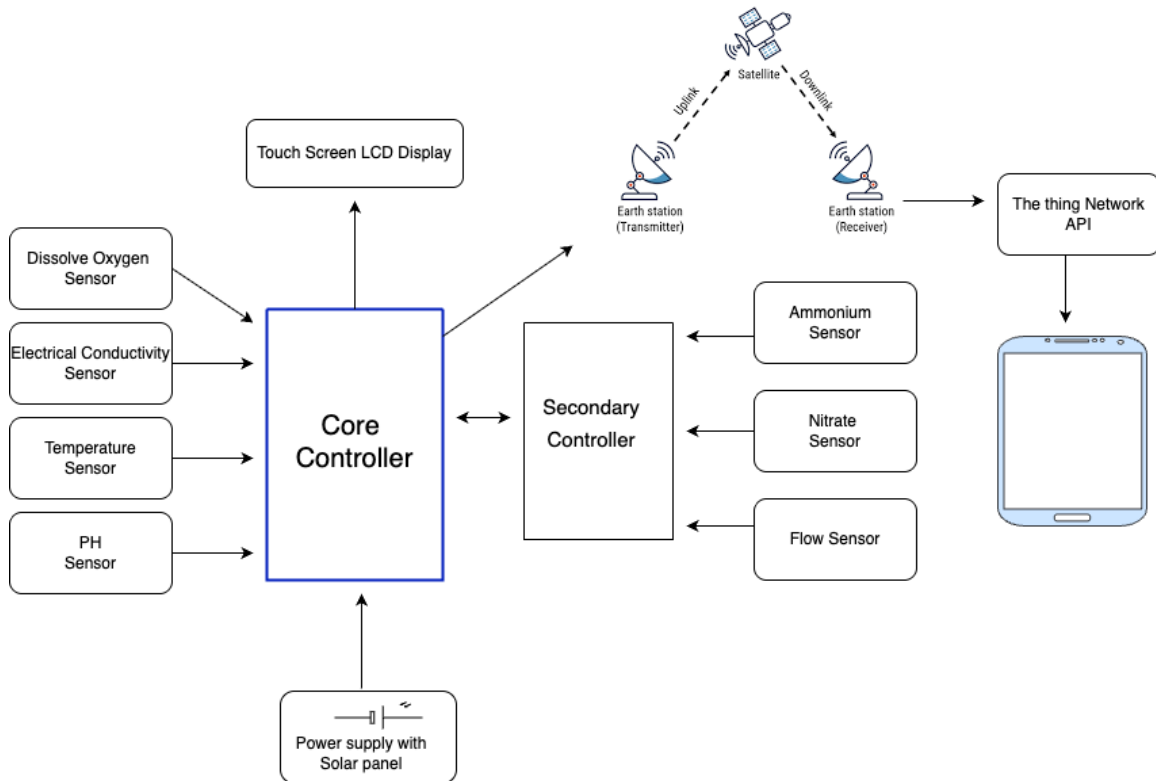


Figure 2. Block Diagram of our system

4. Component Descriptions

This section briefly describes the hardware components used in the EMWAC prototype.

TFT LCD Display (320 x 240 IPS)



Figure 3. TFT display

A 2.0-inch IPS TFT display with a 320 x 240 pixel resolution

(RGB565) is used for on-site status indication and diagnostics. It is driven by an ST7789V controller and supports a four-wire SPI interface as well as a GDI port. The module operates from 3.3–5.5 V and incorporates an onboard MicroSD card slot, which can be used for extended local logging or displaying pre-rendered graphics. Typical full-screen current consumption is about 29 mA.

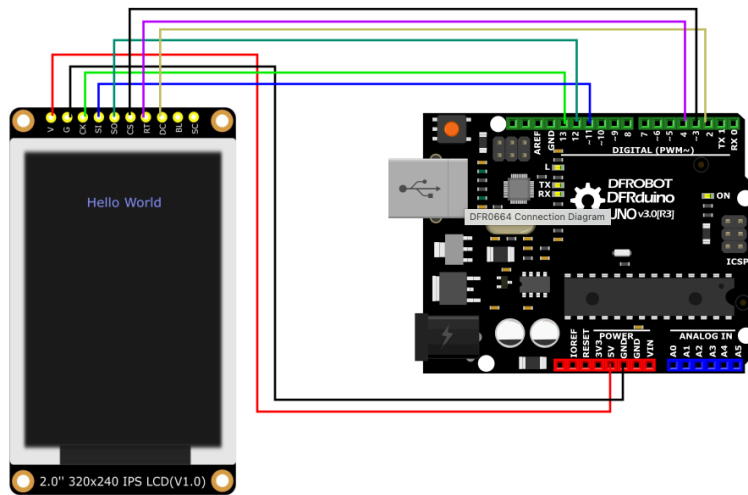


Figure 4. Connection of the display

pH Sensor

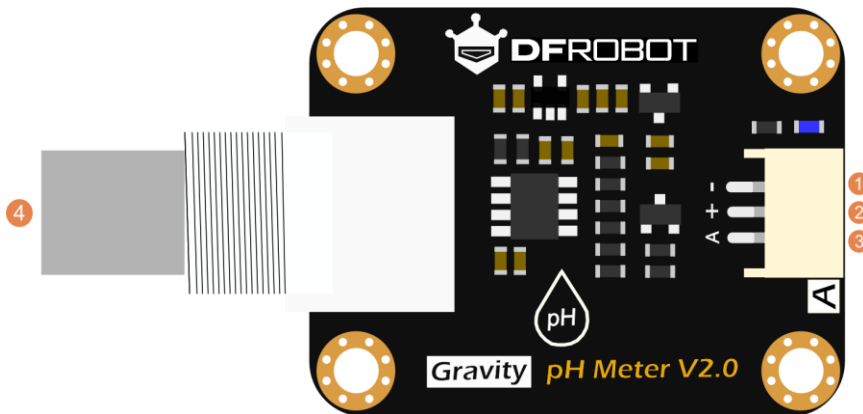


Figure 5. PH sensor board

The pH sensing subsystem is built around an industrial-grade analog pH probe and a Gravity-compatible signal conditioning board. The board includes a voltage regulator that supports 3.3–5.5 V operation, allowing direct connection to a wide range of microcontrollers. Hardware filtering provides a low-noise analog signal (0–3 V), and a two-point calibration method is used in firmware.

Key characteristics:

- Measurement range: pH 0–14
- Accuracy: ± 0.1 pH at 25 °C
- Probe type: industrial, suitable for continuous immersion
- Output: 0–3 V analog via BNC/Gravity connector

The probe requires periodic calibration with pH 4.0 and pH 7.0 buffer solutions to maintain accuracy.

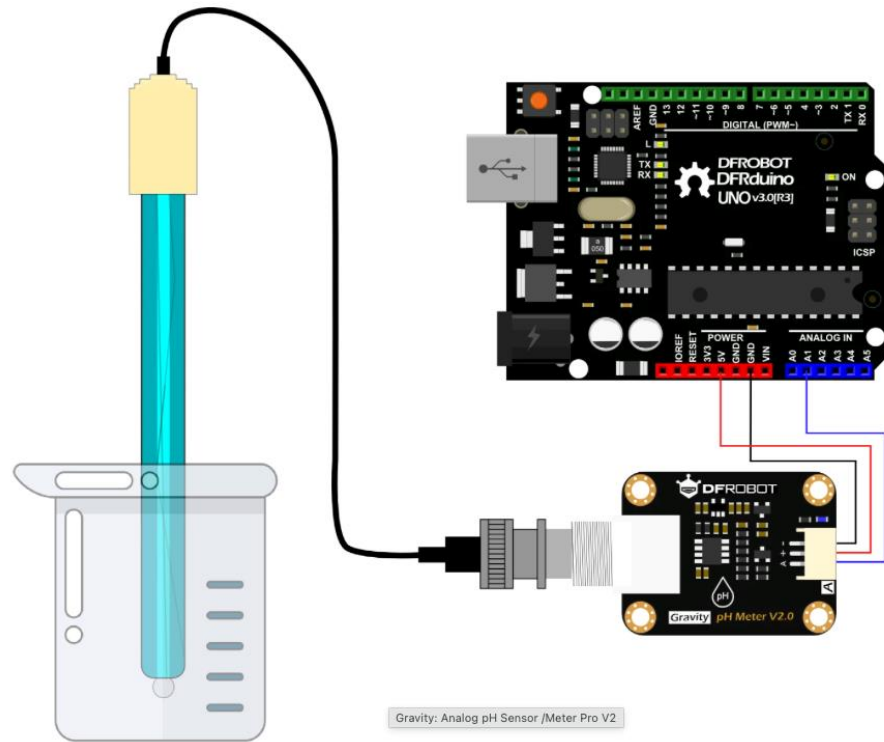


Figure 6. PH sensor connection

Electrical Conductivity (EC) Sensor

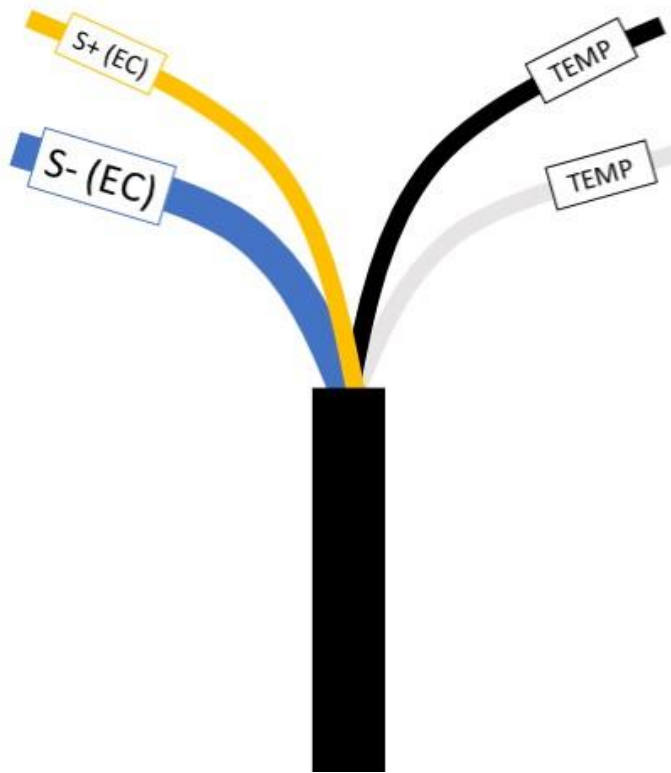


Figure 7. EC sensor probe

The EC sensor measures the conductivity of an aqueous solution, which correlates with ionic concentration and salinity. The industrial probe is rated for continuous immersion and incorporates a PT1000 platinum resistance thermometer for temperature compensation.

Core specifications:

- Supply voltage: 3.3–5.0 V
- Analog output: 0–3 V
- Cell constant: $K = 1$
- Practical range: 100–2000 $\mu\text{S}/\text{cm}$

- Maximum range: 1–2200 $\mu\text{S}/\text{cm}$
- Operating temperature: 0–50 °C (non-freezing)
- Waterproof rating: IP68

The excitation source is an AC signal to minimize electrode polarization and improve probe lifetime.

Dissolved Oxygen (DO) Sensor

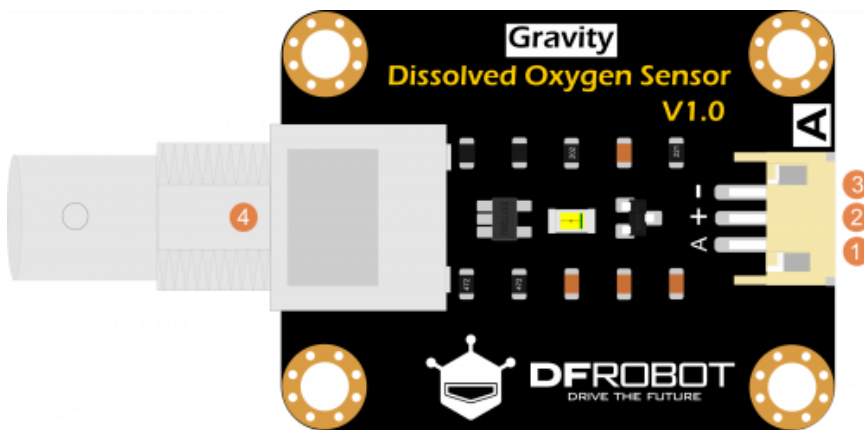


Figure 8. DO sensor board

The dissolved oxygen sensor is a galvanic probe connected to a dedicated signal conversion board. It is used to quantify dissolved oxygen in mg/L, which is critical for aquatic ecosystem health and aquaculture applications.

Main features:

- Type: galvanic probe
- Detection range: 0–20 mg/L
- Response time: up to 98% of final value within 90 seconds at 25 °C
- Operating voltage: 3.3–5.5 V
- Analog output: 0–3 V
- Maintenance: replaceable membrane cap and NaOH filling solution

Proper handling of the NaOH filling solution and the oxygen-permeable membrane is essential for safe and reliable operation.

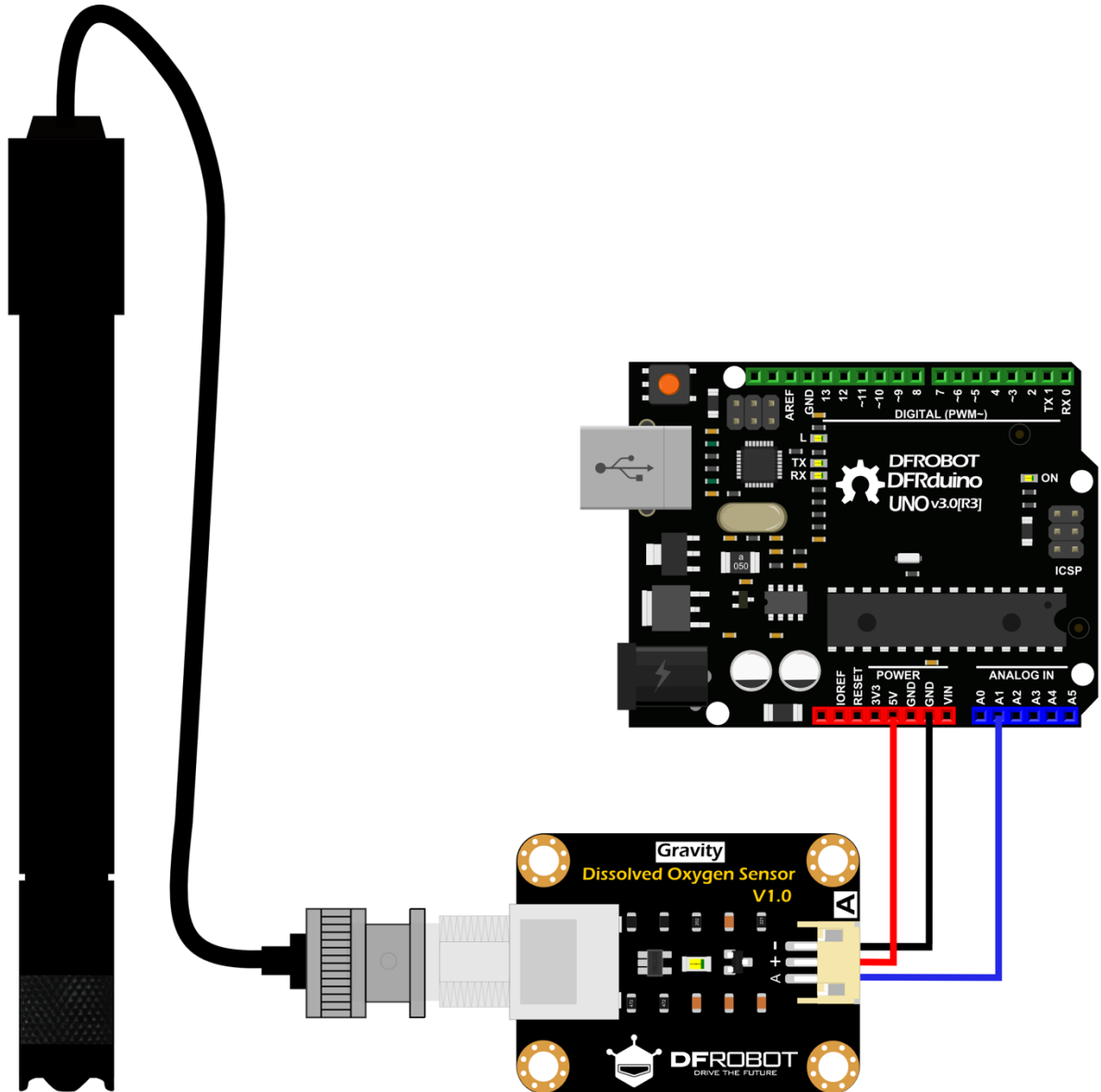


Figure 9. DO sensor connection

Ammonium Ion-Selective Electrode (NH₄⁺ ISE)

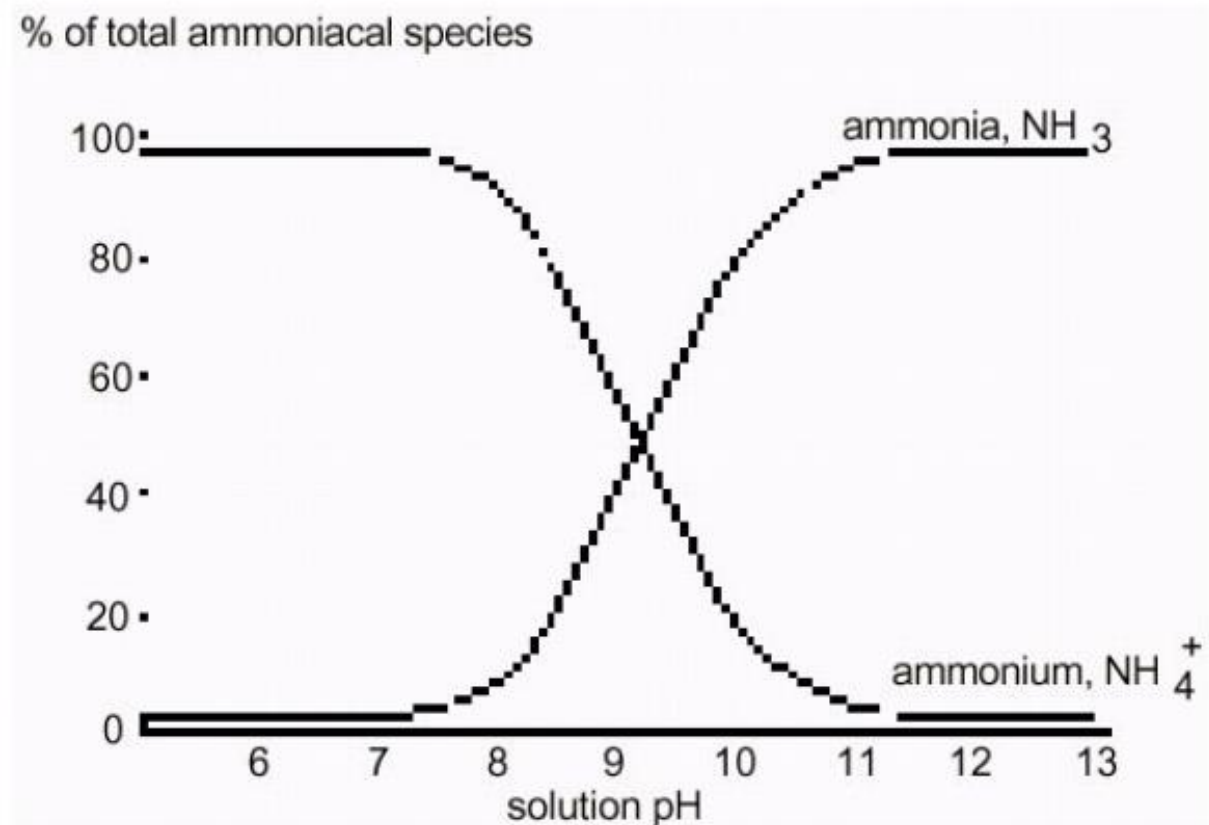


Figure 10. Ammonia, ammonium chart

The ammonium ISE measures NH₄⁺ concentration in water. It is particularly relevant for monitoring agricultural runoff, fertilizer impact, and wastewater.

Key parameters:

- Measurement range: 1–18,000 mg/L
- Typical precision: ±10% of full scale (calibrated 1–100 mg/L)
- Recommended pH range: 4–7.5
- Temperature range: 0–40 °C

The electrode generates a voltage proportional to ion activity in accordance with the Nernst equation. Two-point calibration is generally used, with 1 mg/L and 100 mg/L standards.



Figure 11. Ammonium sensor probe

Nitrate Ion-Selective Electrode (NO₃⁻ ISE)

The nitrate ISE is used to measure nitrate-nitrogen (NO₃⁻ as N), which is an important parameter in pollution monitoring and eutrophication studies.

Characteristics:

- Typical calibration range: 1–100 mg/L NO₃⁻ as N
- Electrode slope: $+56 \pm 4$ mV/decade at 25 °C
- Electrode resistance: 1–4 MΩ
- Replaceable membrane module

As with the ammonium ISE, calibration is performed against low and high standard solutions.

Flow Sensor (YF-S201)

A YF-S201 Hall-effect flow sensor is used to monitor water flow rates through pipes or channels. The sensor consists of a turbine and a Hall sensor that outputs a pulse train whose frequency is proportional to the flow rate.

Basic specifications:

- Operating range: 1–30 L/min
- Supply voltage: 3.5–24 V
- Output: open-collector pulse signal

The sensor is typically connected to a digital input pin with interrupt capability for accurate pulse counting.

5. Sensor Calibration Procedures

Accurate sensor calibration is critical for reliable operation of the EMWAC system over long periods. This section summarizes detailed calibration procedures for the major sensors.

pH Sensor Calibration

The pH subsystem uses a two-point calibration method based on pH 4.0 and pH 7.0 buffer solutions. Calibration parameters are stored in non-volatile memory and should be refreshed periodically.

Required items:

- pH 4.0 and pH 7.0 buffer solutions
- Distilled water
- Clean tissues
- Arduino with pH calibration firmware

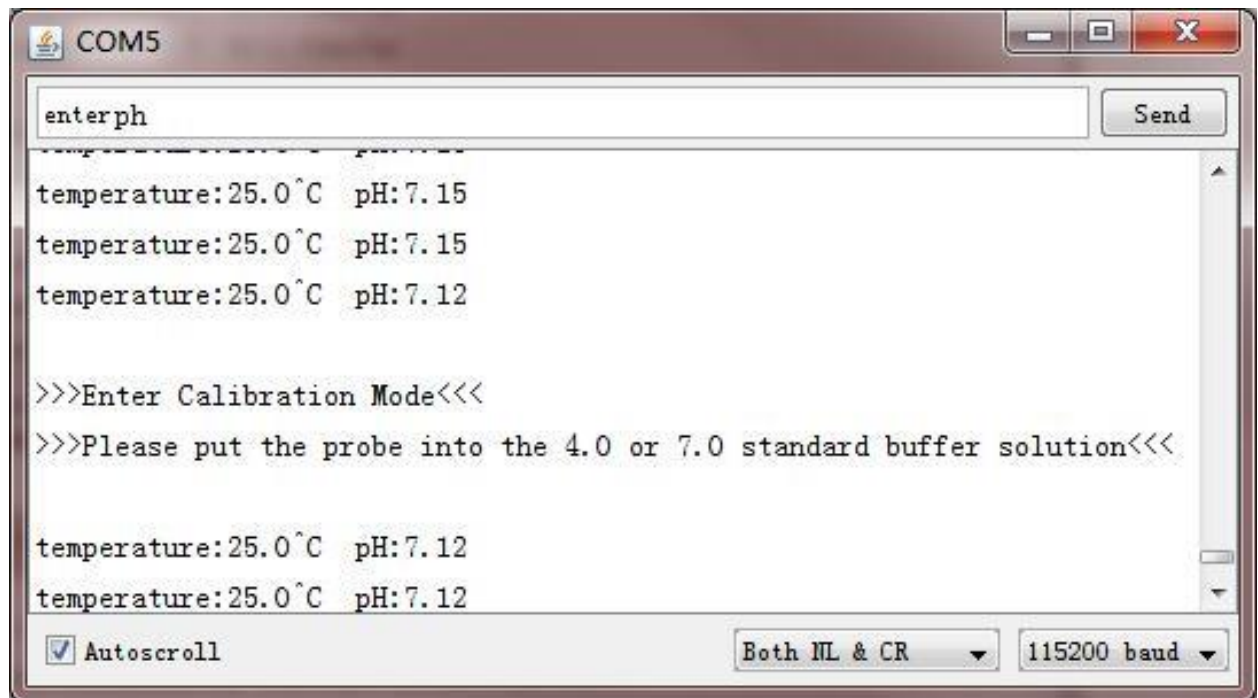
Procedure:

1. Upload the pH calibration sketch to the Arduino and open the serial monitor.
2. Rinse the pH probe with distilled water and gently blot it dry.
3. Immerse the probe in pH 7.0 buffer solution, stir gently, and wait for the reading to stabilize.

4. In the serial monitor, enter the command `enterph` to switch to calibration mode.
5. Enter `calph` to perform the first calibration point. The firmware automatically recognizes the 7.0 buffer.
6. Enter `exitph` to save the calibration parameters to EEPROM.
7. Rinse the probe, blot dry, and immerse it in pH 4.0 buffer solution.
8. Repeat the sequence: `enterph`, `calph`, and `exitph` for the second calibration point.

Notes:

- Perform calibration at least once a month under normal usage, or weekly in demanding applications.
- Use fresh buffer solutions to avoid drift.
- Keep the signal conversion board dry and mechanically isolated from wet surfaces.



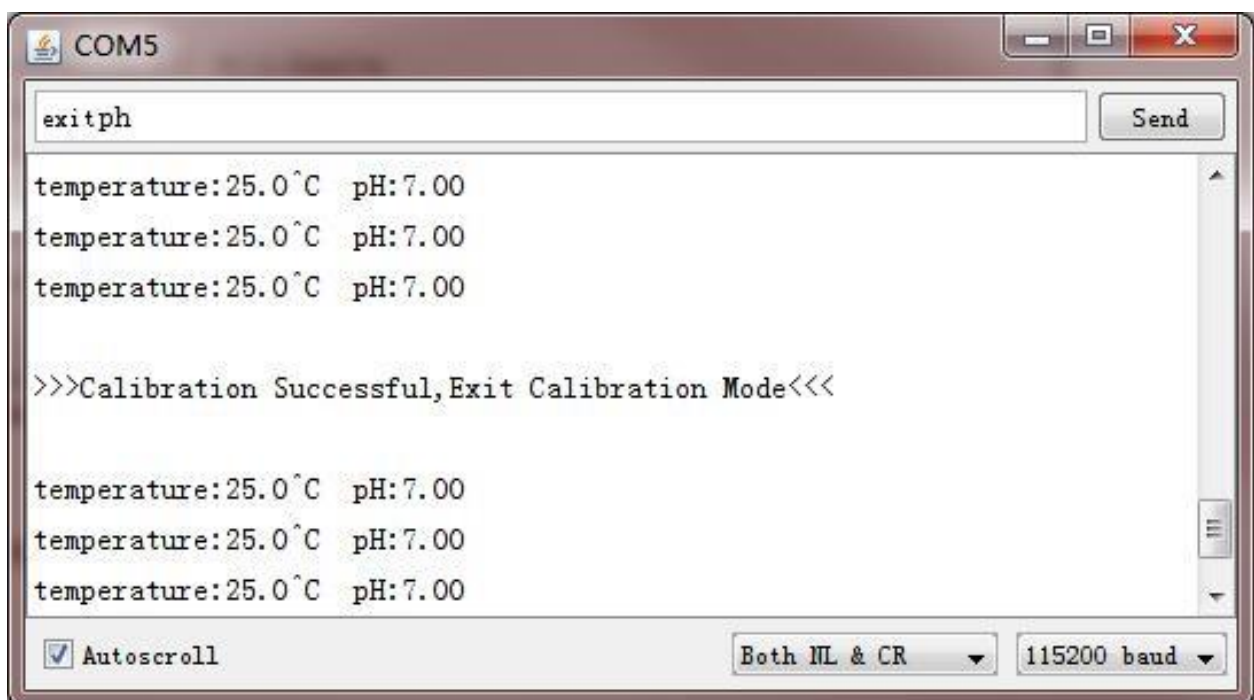
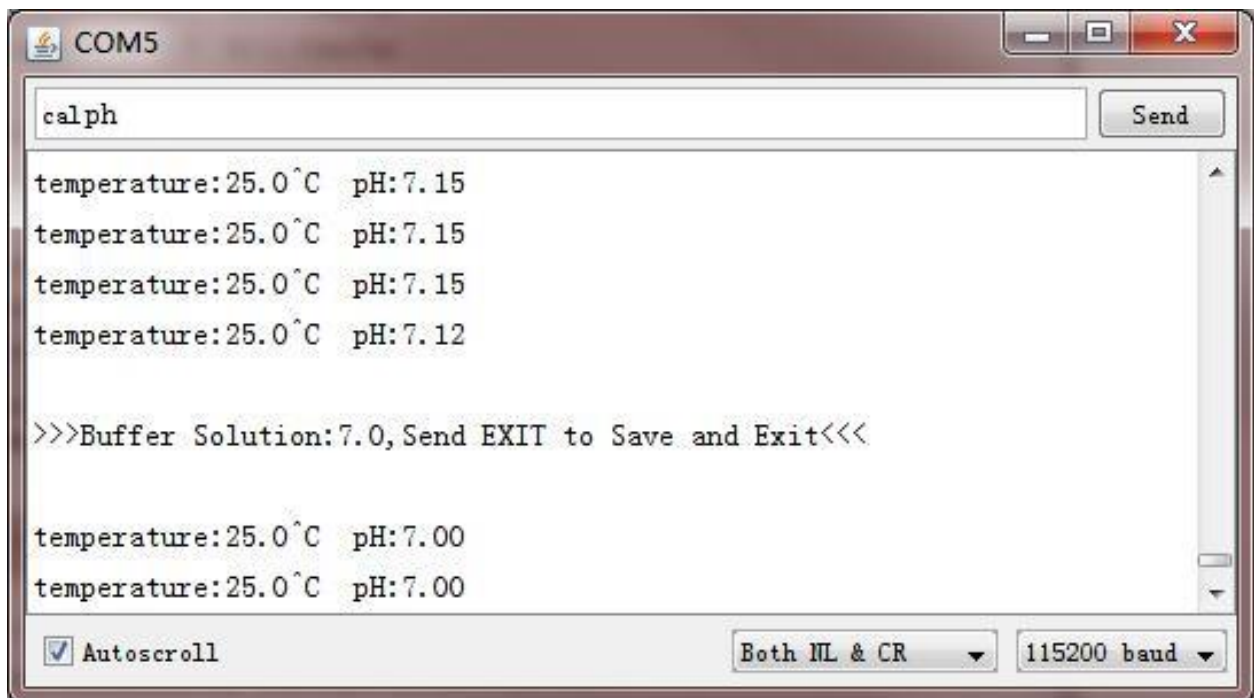


Figure 12. Calibrations of PH Sensor

Electrical Conductivity (EC) Sensor Calibration

The EC sensor is calibrated against a known conductivity standard, commonly 1413 $\mu\text{S}/\text{cm}$. The procedure adjusts the probe's calibration factor K to align measured and reference values.

Required items:

- 1413 $\mu\text{S}/\text{cm}$ conductivity standard solution
- EC probe and conversion board
- PT1000 temperature conversion board
- Arduino with EC calibration firmware

Procedure:

1. Connect the EC probe to the EC conversion board and then to an analog input (A1).
2. Connect the PT1000 board to another analog input (for example A2).
3. Upload the EC measurement sketch and observe the default readings ($K = 1.0$).
4. Immerse the EC probe in the 1413 $\mu\text{S}/\text{cm}$ solution and gently shake to remove bubbles.
5. When the reading stabilizes, enter a command such as `C 1413` in the serial monitor to start calibration.
6. The firmware computes a new K value and prints it to the serial monitor.
7. Update the call to `ec.setCalibration()` in the sketch with the new K value and re-upload the firmware.

Temperature compensation is applied using the PT1000 temperature reading, which is passed into `getEC\_us\_cm()`.

Calibration Program

```
#include "DFRobot_ECPR0.h"

#define EC_PIN A1
#define TE_PIN A2

DFRobot_ECPR0 ec;
```

```

DFRobot_ECPRO_PT1000 ecpt;

uint16_t EC_Voltage, TE_Voltage;
float Conductivity, Temp;

void setup()
{
    Serial.begin(115200);

    ec.setCalibration(1);
    Serial.println("Default Calibration K=" + String(ec.getCalibration()));
    Serial.println("Immerse the probe in the calibration solution and enter \"C  

+ conductivity\" to calibrate");
    Serial.println("e.g. \"C 1413\"");

    delay(2000);
}

void loop()
{
    EC_Voltage = (uint32_t)analogRead(EC_PIN) * 5000 / 1024;
    //TE_Voltage = (uint32_t)analogRead(TE_PIN) * 5000 / 1024;
    TE_Voltage = 783;

    Temp = ecpt.convVoltageToTemperature_C((float)TE_Voltage / 1000);
    Conductivity = ec.getEC_us_cm(EC_Voltage, Temp);

    Serial.print("EC_Voltage: " + String(EC_Voltage) + " mV\t");
    Serial.print("Conductivity: " + String(Conductivity) + " us/cm\t");
    Serial.print("TE_Voltage: " + String(TE_Voltage) + " mV\t");
    Serial.println("Temp: " + String(Temp) + " °C");

    delay(1000);
}

void serialEvent()
{
    String args = Serial.readStringUntil('\n');
    if (args[0] == 'C')
    {
        args.remove(0, 2);
        float calSolution=args.toFloat();
        ec.setCalibration(ec.calibrate(EC_Voltage,calSolution));
        Serial.print("calSolution: " + String(calSolution)+"\t");
        Serial.println("calK: "+ String(ec.getCalibration(),6));
        delay(5000);
    }
    else
        Serial.println("command error");
}

```

Dissolved Oxygen (DO) Sensor Calibration

The DO sensor requires correct filling with NaOH solution and voltage-based calibration.

Filling the probe:

1. Carefully unscrew the membrane cap.
2. Fill approximately two-thirds of the cap with 0.5 mol/L NaOH solution.
3. Hold the probe vertically and slowly screw the cap back on, allowing a small amount of solution to overflow.
4. Wipe away any excess solution and ensure there are no large bubbles inside the cap.

Single-point calibration (saturated DO at a fixed temperature):

1. Wet the probe in pure water and shake off excess drops.
2. Either expose the wet probe to air or immerse it in oxygen-saturated water prepared by vigorous stirring or aeration.
3. After the output voltage stabilizes, record this value as the saturated DO voltage at the current temperature.

Two-point calibration (with temperature compensation):

1. Prepare two beakers of purified water: one cooled and one warmed (below 40 °C).
2. Saturate each with oxygen by stirring or aeration, then let bubbles dissipate.
3. Place the probe in the first beaker, gently stir, and record the stabilized voltage and temperature.
4. Repeat the measurement in the second beaker.
5. Use these two points to derive a temperature compensation curve for saturated DO voltage.

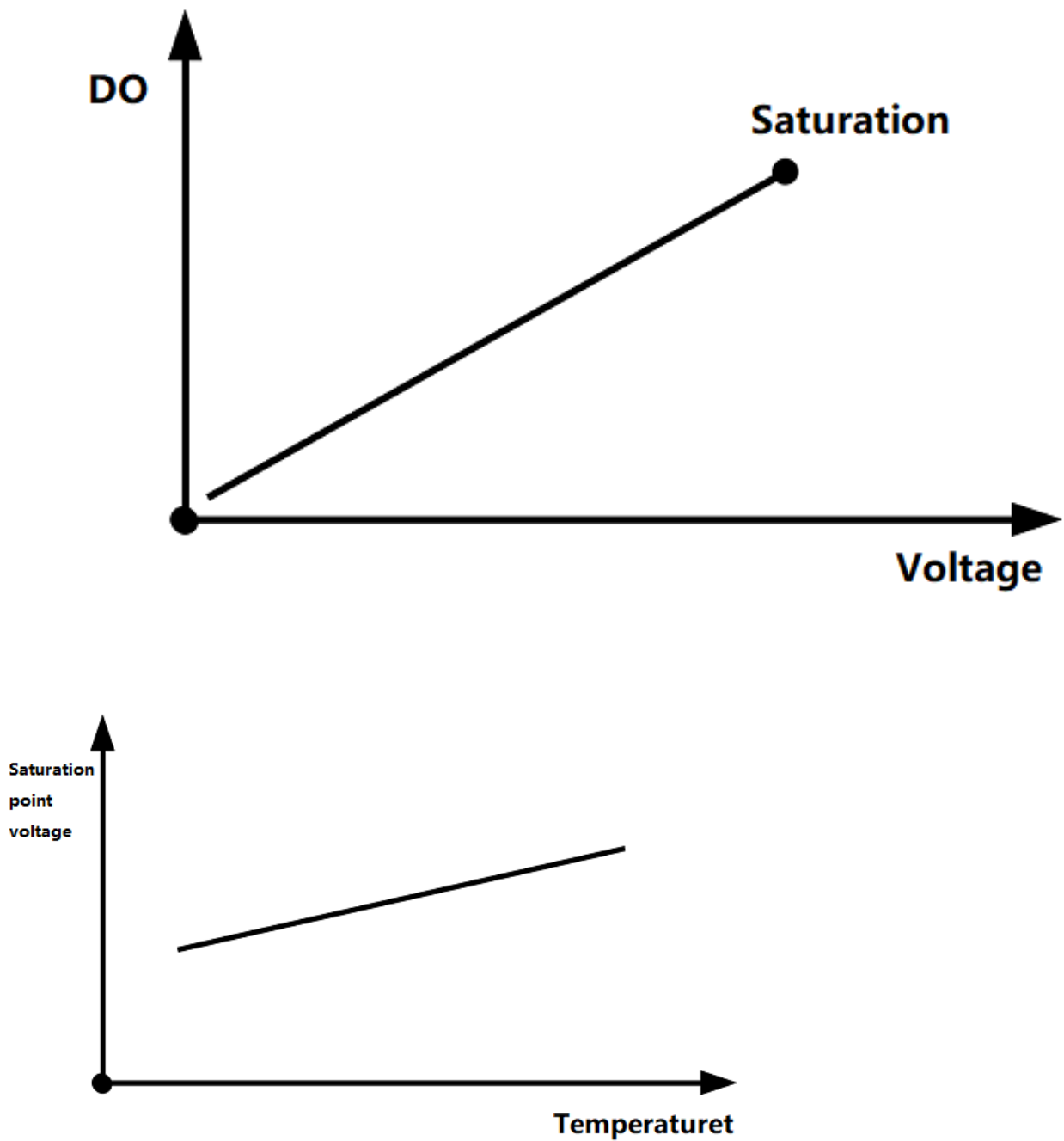


Figure 13. DO calibration chart

Saturated dissolved oxygen levels vary significantly with temperature, which directly influences both the actual oxygen concentration in

water and the sensor's saturation voltage output. To achieve higher measurement accuracy, these temperature-dependent effects must be accounted for during calibration.

The figure below illustrates the general relationship between saturation voltage and temperature. By recording the probe's saturation voltage at two distinct temperatures, a temperature compensation curve can be established.

Since the relationship between temperature and saturated dissolved oxygen at standard atmospheric pressure is well characterized, the system can determine the expected saturated oxygen concentration and its corresponding voltage based on the measured water temperature. This enables accurate dissolved oxygen calculations across varying environmental conditions.

T °C	DO mg/L	T °C	DO mg/L	T °C	DO mg/L
0	14.60	16	9.86	32	7.30
1	14.22	17	9.64	33	7.17
2	13.80	18	9.47	34	7.06
3	13.44	19	9.27	35	6.94
4	13.08	20	9.09	36	6.84
5	12.76	21	8.91	37	6.72
6	12.44	22	8.74	38	6.60
7	12.11	23	8.57	39	6.52
8	11.83	24	8.41	40	6.40
9	11.56	25	8.25	41	6.33
10	11.29	26	8.11	42	6.23
11	11.04	27	7.96	43	6.13
12	10.76	28	7.83	44	6.06
13	10.54	29	7.68	45	5.97
14	10.31	30	7.56	46	5.88
15	10.06	31	7.43	47	5.79

Table of relationship between DO and Temperature

Calibration Code

```
#include <Arduino.h>

#define VREF    5000//VREF(mv)
#define ADC_RES 1024//ADC Resolution

uint32_t raw;

void setup()
{
    Serial.begin(115200);
}

void loop()
{
    raw=analogRead(A1);

    Serial.println("raw:\t"+String(raw)+"\tVoltage(mv)"+String(raw*VR
EF/ADC_RES));
    delay(1000);
}
```

Ammonium ISE Calibration

The ammonium ISE is calibrated using two standard solutions, commonly 1 mg/L and 100 mg/L NH_4^+ as N.

Procedure:

1. Soak the electrode in the high standard (100 mg/L) for approximately 30 minutes, ensuring the reference contacts are immersed.
2. With the electrode still in the high standard, configure your meter or firmware to treat the reading as 100 mg/L and store the calibration point.

3. Rinse the electrode with distilled water and blot dry.
4. Place the electrode in the low standard (1 mg/L), wait for the reading to stabilize, and store this as the second calibration point.

For improved accuracy at low concentrations, add an ionic strength adjuster (ISA) to both standards and samples, typically at a 1:50 ratio (1 mL ISA to 50 mL sample).

Nitrate ISE Calibration

The nitrate ISE is calibrated in a similar way to the ammonium ISE, using 1 mg/L and 100 mg/L nitrate-nitrogen standards.

Procedure:

1. Soak the electrode in the 100 mg/L standard for about 30 minutes.
2. With the electrode in the high standard, store the reading as 100 mg/L NO₃⁻ as N.
3. Rinse, blot dry, and immerse the electrode in the 1 mg/L standard.
4. Store the reading as 1 mg/L NO₃⁻ as N after stabilization.

If the sensor response becomes sluggish or the calibration voltage range changes significantly, the membrane module should be replaced.

6. Wiring and Integration

Proper wiring and integration are essential to ensure stable and noise-resistant measurements from all sensors. The following subsections provide a typical wiring scheme for the EMWAC prototype.

Microcontroller Roles

Arduino UNO R4: primary data acquisition and control of the sensor array, local processing, and user interface updates.

Arduino MKR 1500: wide-area communication using NB-IoT/LTE-M and forwarding data to the cloud and satellite network.

General Wiring Guidelines

- Use shielded cables or twisted-pair wiring for low-level analog signals where possible.
- Keep sensor signal cables physically separated from high-current power lines.
- Route cables through waterproof glands in the enclosure to prevent moisture ingress.
- Provide mechanical strain relief for all external cables.

Sensor-to-Arduino Pin Mapping (UNO R4)

The mapping of sensor outputs to Arduino UNO R4 pins. Exact choices may be adapted to suit alternative enclosure layouts or additional peripherals.

Arduino UNO R4 Pin Assignment

pH sensor analog output -> Analog input A0

EC sensor analog output -> Analog input A1

DO sensor analog output -> Analog input A2

Temperature sensor output -> Analog input A3

Ammonium ISE amplifier output -> Analog input A4

Nitrate ISE amplifier output -> Analog input A5

Flow sensor pulse output -> Digital pin D2 (interrupt-capable)

TFT LCD SPI SCK -> SCK pin

TFT LCD SPI MOSI -> MOSI pin

TFT LCD chip select/data-command/reset -> suitable digital pins (Pin D8, D9, D10)

Power Distribution

- The 12 V battery is connected to a power management module that provides regulated 5 V and 3.3 V outputs.
- The Arduino UNO R4 and MKR 1500 are typically powered from the 5 V rail (subject to board requirements).
- Sensors that support 3.3–5.5 V input can share either the 5 V rail or a dedicated 3.3 V regulator as required by noise and stability considerations.
- Common ground (0 V) must be shared across all boards and sensors to ensure valid analog measurements.

Communication Links

- The Arduino UNO R4 and MKR 1500 may communicate using a serial UART, I2C, or SPI link, depending on the chosen firmware architecture.
- For simple designs, a serial UART connection with a defined message protocol is sufficient for transferring processed sensor data from the UNO R4 to the MKR 1500.

Enclosure and Connectors

-
- The electronics are mounted inside a waterproof enclosure. Sensors are connected through panel-mount connectors (such as BNC, M12, or sealed Gravity-compatible jacks).

- Cable glands are used where fixed cables pass through the enclosure wall, ensuring IP-rated sealing.

7. Testing and Deployment

Once the hardware is assembled and calibrated, the system should be tested in stages before full deployment in the field.

Bench Testing

1. Power the system from a laboratory supply or the intended battery pack.
2. Confirm that the Arduino UNO R4 and MKR 1500 boot correctly and that the TFT LCD shows basic system status.
3. Verify that each analog sensor returns a plausible baseline reading in a known reference solution or clean water.
4. Confirm that the flow sensor outputs pulses when water passes through it and that the firmware converts pulse counts to flow rate.
5. Exercise the communication path by sending a set of sample measurements through the MKR 1500 and confirming reception on the cloud platform.

Field Testing

1. Deploy the enclosure in the intended environment and secure all cables and probes.
2. Allow the sensors to equilibrate in the water for an appropriate settling period.
3. Compare EMWAC readings with reference measurements taken using handheld meters or laboratory analysis where possible.
4. Monitor data over several days to validate long-term stability, power consumption, and communication reliability.

Maintenance

- Perform sensor calibration at the recommended intervals for each sensor type.
- Inspect cables, connectors, and seals for signs of mechanical wear or water ingress.
- Replace consumable components such as DO membranes, NaOH filling solution, and ISE membranes according to manufacturer guidance.

8. Conclusion

This report has described the design and implementation of the EMWAC water quality monitoring prototype. The system integrates multiple industrial water quality sensors, low-power microcontrollers, and satellite-enabled communication to provide continuous monitoring of key environmental parameters in remote areas.

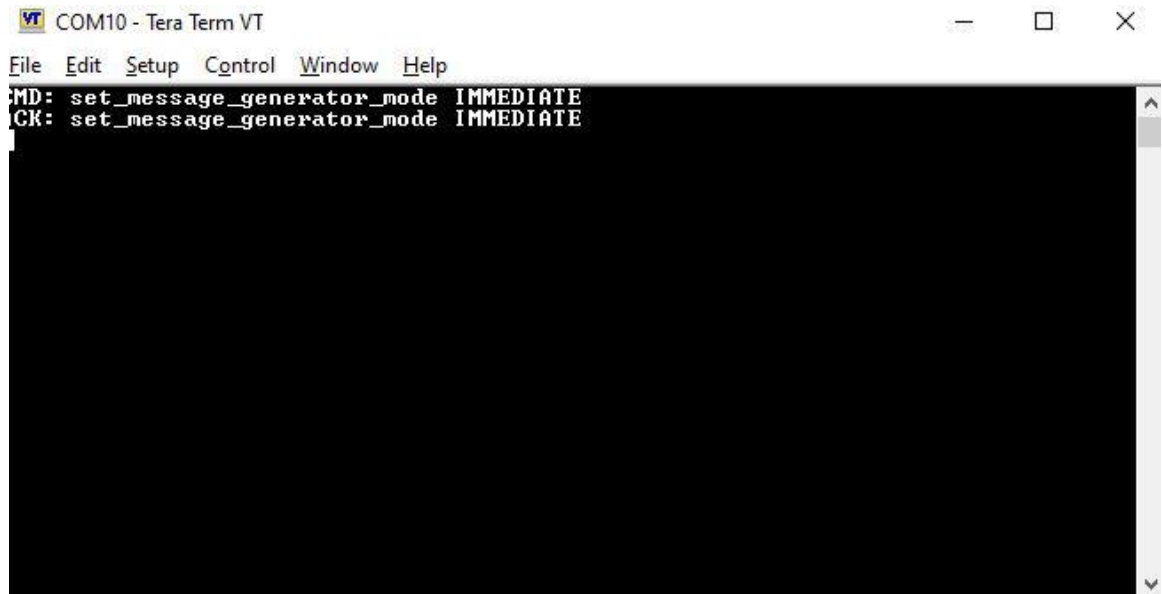
By combining robust hardware architecture, careful calibration procedures, and flexible wiring and integration strategies, the EMWAC platform can serve as a foundation for future deployments and extensions. Potential enhancements include additional sensing parameters, improved power management, and advanced cloud-based analytics for anomaly detection and forecasting.

Appendix: Satellite configuration images

1. Configuration of almanac

```
VT COM10 - Tera Term VT
File Edit Setup Control Window Help
CMD: get_software_version
ACK: get_software_version lacuna-software-modem=2.1.1 sensor-application=2.0.2 #
20
CMD: get_status
LOG: W MainApp: no country set, please set country
NAK: get_status
CMD: set_country GBR
ACK: set_country
EVT: sat_uplink_contact_refreshed
CMD: set_device_address 27FCE0EC
LOG: W SatMAG: init_session: could not recover session
ACK: set_device_address
CMD: set_key SAT_MWKS_0 429C7C5BD0CBFF884CF9D983D0990985
ACK: set_key
CMD: set_key SAT_APPS_0 5A5BC874259359E75B3FD97C8174029C
ACK: set_key
CMD: clear_workspace
ACK: clear_workspace
CMD: write_workspace --offset=0 01de53f6b576fe1a8394a76af2af0f344b727f26653c4c466695c3fb4af2de42
7e143b2a903c098f494c598895cb8cdbe8f32bfbcbcd00d5f7f5afda264b63388e71467cfc700689649360000ff68a0d5
3668a8be3668a8be36ffff0000000000000000201000000012868947157c28b6d1b45aed8c7ba9a104c42c360903898
7a7145a33efa4049613345aec727bfe8aeae3981e6ea200000000061c68940213c1aa362746abfd05b70d29044233e595
3899063c4673d3a6411b4b1346ac01504026790eb9f23a6d03002f5e3f082f7f3f3f0a2f7f3f1b00000001660000000100
00000000fffffb00000000166000000010000000000fffffb100000000164
ACK: write_workspace
CMD: write_workspace --offset=255 000000010000000000fffffc6000000014200000002000000001fffff64
0000000186000000200000000002fffff84000000018200000020000000002fffffe4000000018000000020000000
0002fffff4c00000001060000000000000002fffffa500000001c200000010000000003fffff68000000060300
0001000000000154ffff0ae40000000603000020000000000154ffff0be4000000060300000000000000154ffff06d6
00000006030000000000000000154ffff00e400000006010000000000000154ffff004c000000060100000400000000
0154ffff014c000000060100000200000000154ffff024c0000000601000010
ACK: write_workspace
CMD: write_workspace --offset=510 000000000154ffff034c000000060100000400000000154ffff044c000000
060100000200000000154ffff054c0000000603000000400000000154ffff07e40000000603000000400000000154ff
ff08e40000000605000001000000000154ffff09aa000000060700000100000000154ffff0910000000060500002000
0000000154ffff0d4f0000000601000010000000000154ffff0eaa0000000601000001000000001540000c24000000
0605000020000000000154ffff0f420e436607d043c007d047901770479f177047ff07d04830177097be1025897be70258
9ce902589cf502589cf502589cfe02589d0102589d04025801abc60600a70410
ACK: write_workspace
CMD: write_workspace --offset=765 006400a7041b006400c3042000640097042000640097031e0064017b01003e
00005a4c71de204010eb0004010fcbfd00000f002ef0071670f00f5b0304ff00b00000feb0f00e000000f00f00f0e00f
0e0004d00f0f04000f2000f00f00fffb0dd0000fffb0b0bf0b000000f00f00dd0f0c718c419f0e00eb41111b2d4f0010dd
9ade1946dd4f2ff4546c04cf9dc46d53af21646c317f5a246dcafee8846ff4a495846c630343246aae1447446275c2379
46b9b1b3946af4a266646a0ad94646c7320c2546cb17063a46065456c1460001004d00
ACK: write_workspace
CMD: install_almanac --expected_size=976
LOG: W LPP: storage_write: restore_session(LPP2Data0)
ACK: install_almanac
EVT: sat_uplink_contact_refreshed
█
```

2. Message generation config



A screenshot of a Tera Term VT window titled "COM10 - Tera Term VT". The window has a menu bar with "File", "Edit", "Setup", "Control", "Window", and "Help". The terminal content shows two lines of commands: "CMD: set_message_generator_mode IMMEDIATE" and "ACK: set_message_generator_mode IMMEDIATE". The rest of the terminal area is black.

```
VT COM10 - Tera Term VT
File Edit Setup Control Window Help
CMD: set_message_generator_mode IMMEDIATE
ACK: set_message_generator_mode IMMEDIATE
```

3. Setting device address

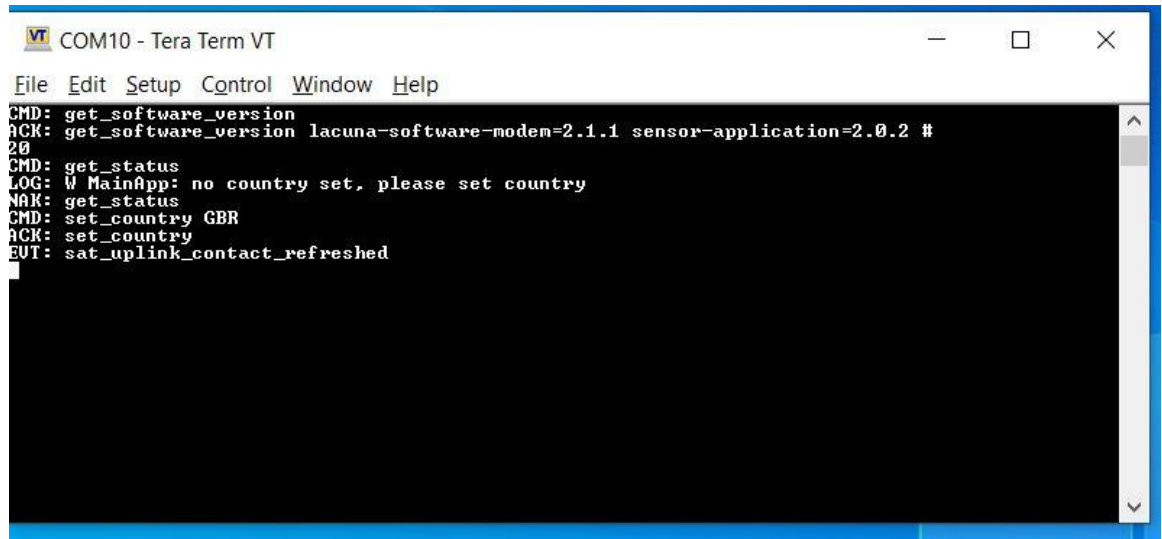


A screenshot of a Tera Term VT window titled "COM10 - Tera Term VT". The window has a menu bar with "File", "Edit", "Setup", "Control", "Window", and "Help". The terminal content shows a sequence of commands and responses: "CMD: get_software_version", "ACK: get_software_version lacuna-software-modem=2.1.1 sensor-application=2.0.2 # 20", "CMD: get_status", "LOG: W MainApp: no country set, please set country", "NAK: get_status", "CMD: set_country GBR", "ACK: set_country", "EUT: sat_uplink_contact_refreshed", "CMD: set_device_address 27FCE0EC", "LOG: W SatMAC: init_session: could not recover session", and "ACK: set_device_address". The rest of the terminal area is black.

```
VT COM10 - Tera Term VT
File Edit Setup Control Window Help
CMD: get_software_version
ACK: get_software_version lacuna-software-modem=2.1.1 sensor-application=2.0.2 #
20
CMD: get_status
LOG: W MainApp: no country set, please set country
NAK: get_status
CMD: set_country GBR
ACK: set_country
EUT: sat_uplink_contact_refreshed
CMD: set_device_address 27FCE0EC
LOG: W SatMAC: init_session: could not recover session
ACK: set_device_address
```

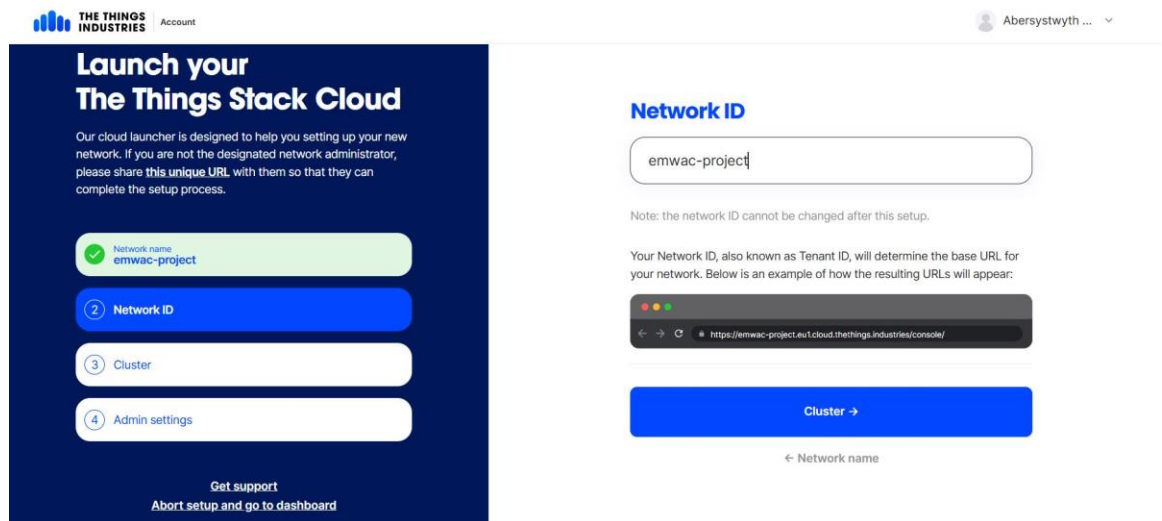
[illegible]

6. Set Country



```
COM10 - Tera Term VT
File Edit Setup Control Window Help
CMD: get_software_version
ACK: get_software_version lacuna-software-modem=2.1.1 sensor-application=2.0.2 #
20
CMD: get_status
LOG: W MainApp: no country set, please set country
NAK: get_status
CMD: set_country GBR
ACK: set_country
EUT: sat_uplink_contact_refreshed
```

7. The following images are the configuration of the thing network TTN



Launch your The Things Stack Cloud

Our cloud launcher is designed to help you setting up your new network. If you are not the designated network administrator, please share [this unique URL](#) with them so that they can complete the setup process.

- 1 Network name: emwac-project
- 2 Network ID
- 3 Cluster
- 4 Admin settings

[Get support](#)
[Abort setup and go to dashboard](#)

Network ID

emwac-project

Note: the network ID cannot be changed after this setup.

Your Network ID, also known as Tenant ID, will determine the base URL for your network. Below is an example of how the resulting URLs will appear:

<https://emwac-project.eu1.cloud.thethingsindustries/console/>

[Cluster ->](#)

[← Network name](#)

Launch your The Things Stack Cloud

Our cloud launcher is designed to help you setting up your new network. If you are not the designated network administrator, please share [this unique URL](#) with them so that they can complete the setup process.

✓ Network name
emwac-project

✓ Network ID
emwac-project

3 Cluster

4 Admin settings

Cluster

Select...

Choose the [deployment region](#) that corresponds to the location where your end devices will primarily operate. This will help us optimize your experience by localizing your network, and enable optional [peering capabilities](#).

Administrator login →

← Network ID

THE THINGS INDUSTRIES Account

Abersystwyth ...

Launch your The Things Stack Cloud

Our cloud launcher is designed to help you setting up your new network. If you are not the designated network administrator, please share [this unique URL](#) with them so that they can complete the setup process.

✓ Network name
emwac-project

✓ Network ID
emwac-project

3 Cluster

4 Admin settings

[Get support](#)
[Abort setup and go to dashboard](#)

Cluster

Europe 2 (London, UK)

Choose the [deployment region](#) that corresponds to the location where your end devices will primarily operate. This will help us optimize your experience by localizing your network, and enable optional [peering capabilities](#).

Administrator login →

← Network ID

Launch your The Things Stack Cloud

Our cloud launcher is designed to help you setting up your new network. If you are not the designated network administrator, please share [this unique URL](#) with them so that they can complete the setup process.

✓ Network name
emwac-project

✓ Network ID
emwac-project

✓ Cluster
Europe 2 (London, UK)

4 Admin settings

[Get support](#)
[Abort setup and go to dashboard](#)

Admin settings

Set up the administrator account on your new The Things Stack Cloud. This account is used when signing into the [Console](#) or [CLI](#). After launching your network, you will receive an email with a link to set up your password.

Full name

Email

Repeat email

Username

Launch your The Things Stack Cloud

Our cloud launcher is designed to help you setting up your new network. If you are not the designated network administrator, please share [this unique URL](#) with them so that they can complete the setup process.

✓ Network name
emwac-project

✓ Network ID
emwac-project

✓ Cluster
Europe 2 (London, UK)

4 Admin settings

[Get support](#)
[Abort setup and go to dashboard](#)

Admin settings

Set up the administrator account on your new The Things Stack Cloud. This account is used when signing into the [Console](#) or [CLI](#). After launching your network, you will receive an email with a link to set up your password.

Full name

Email

Repeat email

Username



Your The Things Stack is ready

We've successfully launched your network server. You can now start using it.

[Go to the Console](#)

[See my subscriptions](#)

[Read our getting started guide](#) [Install Command-line interface](#)

THE THINGS STACK
CLOUD DISCOVERY

HomeApplicationsGateways

SearchCtrlK

EMWAC Project Application

Application overviewEnd devicesLive dataWebhooksMessage storageNetwork Operations CenterPayload formatters

The Things Stack Cloud Discovery0 of 10 devices0 of 10 gatewaysUpgrade now

Resourceseu2 • v3.34.2.Beb6d17445

Applications > EMWAC Project Application > End devices > Register end device

Degraded performanceAdd

Register end device

Does your end device have a LoRaWAN® Device Identification QR Code? Scan it to speed up onboarding.

Scan end device QR codeDevice registration help

End device type

Input method

- Select the end device in the LoRaWAN Device Repository
- Enter end device specifics manually

Frequency plan *Europe 863-870 MHz (SF9 for RX2 - recommended)

LoRaWAN version *LoRaWAN Specification 1.0.4

Regional Parameters version *RP002 Regional Parameters 1.0.3

Show advanced activation, LoRaWAN class and cluster settings

THE THINGS STACK
CLOUD DISCOVERY

HomeApplicationsGateways

SearchCtrlK

EMWAC Project Application

Application overviewEnd devicesLive dataWebhooksMessage storageNetwork Operations CenterPayload formatters

The Things Stack Cloud Discovery0 of 10 devices0 of 10 gatewaysUpgrade now

Resources

Applications > EMWAC Project Application > End devices > Register end device

Degraded performanceAdd

Network defaults

- Use network's default MAC settings

Cluster settings

- Skip registration on Join Server

Provisioning information

DevEUI *78 B3 D5 7E D8 00 4A 14Generate1/50 used

Device address *27 FC E0 ECGenerate

AppSKey *5A 5B C8 74 25 93 59 E7 5B 3F D9 7C 81 74 02 9CGenerate

NwkSKey *42 9C 7C 5B D0 C8 FF 88 4C F9 D9 83 D0 99 09 85Generate

End device ID *emwac-satellite

THE THINGS STACK
CLOUD DISCOVERY

HomeApplicationsGateways

SearchCtrlK

EMWAC Project Application

Application overview

End devices

Live data

Webhooks

Message storage

Network Operations Center

Payload formatters

The Things Stack Cloud Discovery

0 of 10 devices

0 of 10 gateways

Upgrade now

Resourceseu2 • v3.34.2.8eb6d17445

Applications > EMWAC Project Application > End devices > Register end device

Degraded performanceAdd

Device address*27 FC E0 ECGenerate

AppSKey*5A 5B C8 74 25 93 59 E7 58 3F D9 7C 81 74 02 9CGenerate

NwkSKey*42 9C 7C 5B D0 CB FF 88 4C F9 D9 83 D0 99 09 85Generate

End device ID*emwac-satellitd

Device properties

Labels

Select...

Add a label or create one to categorize your devices

After registration

View registered end device

Register another end device of this type

Register end device

THE THINGS STACK
CLOUD DISCOVERY

HomeApplicationsGateways

SearchCtrlK

EMWAC Project Application

Application overview

End devices

Live data

Webhooks

Message storage

Network Operations Center

Payload formatters

The Things Stack Cloud Discovery

0 of 10 devices

0 of 10 gateways

Upgrade now

Resourceseu2 • v3.34.2.8eb6d17445

Applications > EMWAC Project Application > End devices > Register end device

Degraded performanceAdd

Show advanced activation, LoRaWAN class and cluster settings ^

Activation mode

Over the air activation (OTAA)

Activation by personalization (ABP)

Define multicast group (ABP & Multicast)

Additional LoRaWAN class capabilities

None (class A only)

Network defaults

Use network's default MAC settings

Rx1 data rate offset*1secResets frame counters

0Enabled

Rx2 data rate*869.525MHz

Factory preset frequencies

+ Add entry

Cluster settings

Skip registration on Join Server

THE THINGS STACK

CLOUD DISCOVERY

Home

Applications

Gateways

Search

Ctrl

K

EMWAC Project Application

Application overview

End devices

Live data

Webhooks

Message storage

Network Operations Center

Payload formatters

The Things Stack Cloud Discovery

0 of 10 devices

0 of 10 gateways

Upgrade now

Resources

eu2 • v3.34.2.8eb6d17445

Applications > EMWAC Project Application > End devices > emwac-satellite > Device overview

Degraded performance

+ Add

☆

🔍

📄

👤

⌵

emwac-satellite

ID: emwac-satellite

+ Add label

No activity yet • 14 n/a up / n/a down

☆

☰

Device overview

Live data

Messaging

Location

Payload formatters

Settings

End device info

Device repository →

emwac-satellite

This end device is currently not part of our Device Repository

Latest decoded payload

See in live data →

No recent activity

There haven't been any uplinks recently in this application

General information

End device ID

emwac-satellite

Frequency plan

Europe 863-870 MHz (SF9 for RX2 - recommended)

LoRaWAN version

LoRaWAN Specification 1.0.4

Regional Parameters version

RP002 Regional Parameters 1.0.3

Network activity

NOC →

Packets per data rate

Packets per channel (24 days)

Live data

THE THINGS STACK

CLOUD DISCOVERY

Home

Applications

Gateways

Search

Ctrl

K

EMWAC Project Application

Application overview

End devices

Live data

Webhooks

Message storage

Network Operations Center

Payload formatters

Uplink

Downlink

Collaborators

API keys

Other integrations

General settings

Top end devices

emwac-satellite

The Things Stack Cloud Discovery

1 of 10 devices

0 of 10 gateways

Upgrade now

Applications > EMWAC Project Application > Payload formatters > Uplink

+ Add

☆

🔍

📄

👤

⌵

EMWAC Project Application

ID: emwac-app1.local.com

+ Add label

Last activity 4 days ago • 1 End devices

☆

☰

Default uplink payload formatter

You can use the "Payload formatter" tab of individual end devices to test uplink payload formatters and to define individual payload formatter settings per end device.

Setup

Formatter type*

Custom Javascript formatter

Formatter code*

```
function decodeUplink(input) {
  return {
    data: {
      bytes: input.bytes
    },
    warnings: [],
    errors: []
  };
}
```

Save changes

New deployment

Deployment successfully updated.

Version 1 on Oct 6, 2025, 11:04 AM

Deployment ID

AKfycbzGxOom2i51-cVo0P8Mxltymb0M8Ho-2Tjv0qcccYE03rE3KNcgyzNMwtJp_Yog_iz_

 [Copy](#)

Web app

URL

https://script.google.com/macros/s/AKfycbzGxOom2i51-cVo0P8Mxltymb0M8Ho-2Tjv0qcccYE03rE3KNcgyzNMwtJp_Yog...

 [Copy](#)

Done

Sensor codes.

```
#include <Arduino.h>
```

```
#include "DFRobot_GDL.h"
```

```
#include "DFRobot_PH.h"
```

```
#include "DFRobot_ECPRO.h"
```

```
#if defined(ARDUINO_SAM_ZERO)

#define TFT_DC 7

#define TFT_CS 5

#define TFT_RST 6

#elif defined(ESP32) || defined(ESP8266)

#define TFT_DC D2

#define TFT_CS D6

#define TFT_RST D3

#else

#define TFT_DC 2

#define TFT_CS 3

#define TFT_RST 4

#endif
```

```
DFRobot_ILI9488_320x480_HW_SPI screen(TFT_DC, TFT_CS,
TFT_RST);
```

```
#define PH_PIN A1

#define DO_PIN A2

#define EC_PIN A3

#define TE_PIN A5
```

```
#define VREF_MV 5000UL
```

```
#define ADC_RES 1024UL
```

```
#define TWO_POINT_CALIBRATION 0
```

```
#define READ_TEMP_C 16
```

```
#define CAL1_V 4736
```

```
#define CAL1_T 16
```

```
#define CAL2_V 1300
```

```
#define CAL2_T 16
```

```
const uint16_t DO_Table[41] = {
```

```
14460,14220,13820,13440,13090,12740,12420,12110,11810,11530,
```

```
11260,11010,10770,10530,10300,10080,9860,9660,9460,9270,
```

```
9080,8900,8730,8570,8410,8250,8110,7960,7820,7690,
```

```
7560,7430,7300,7180,7070,6950,6840,6730,6630,6530,6410
```

```
};
```

```
DFRobot_PH ph;
```

```
DFRobot_ECPRO ec;
```

```
DFRobot_ECPRO_PT1000 ecpt;
```

```
float temperatureC = 25.0f;
```

```
float pHValue = 0.0f;
```

```
float DOValue = 0.0f;
```

```
float ECValue = 0.0f;
```

```
float AmmoniumValue = 0.0f;
```

```
float NitrateValue = 0.0f;
```

```
float FlowValue = 0.0f;
```

```
String rxLine;
```

```
static inline uint16_t adcToMillivolts(int adc)
```

```
{
```

```
    return (uint32_t)VREF_MV * (uint32_t)adc / (uint32_t)ADC_RES;
```

```
}
```

```
int16_t readDOmgL(uint32_t voltage_mv, uint8_t temperature_c)
```

```
{
```

```
#if TWO_POINT_CALIBRATION == 0
```

```
    uint16_t V_saturation = (uint32_t)CAL1_V + (uint32_t)35 *  
    temperature_c - (uint32_t)CAL1_T * 35;
```

```
    return (int16_t)(voltage_mv * DO_Table[temperature_c] /  
    V_saturation);
```

```
#else
```

```

    uint16_t V_saturation = (int16_t)((int8_t)temperature_c - CAL2_T) *
    ((uint16_t)CAL1_V - CAL2_V) / ((uint8_t)CAL1_T - CAL2_T) +
    CAL2_V;

    return (int16_t)(voltage_mv * DO_Table[temperature_c] /
    V_saturation);
#endif
}

```

```

float readTemperatureC()
{
    uint16_t te_mv = adcToMillivolts(analogRead(TE_PIN));
    float t = ecpt.convVoltageToTemperature_C((float)te_mv / 1000.0f);
    if (!isnan(t) && t > -20.0f && t < 80.0f) return t - 28.0f;
    return temperatureC;
}

```

```

float readECuscm(float tempC)
{
    uint16_t ec_mv = adcToMillivolts(analogRead(EC_PIN));
    float val = ec.getEC_us_cm(ec_mv, tempC);
    if (!isnan(val) && val >= 0.0f) return val;
    return ecValue;
}

```

```

float readPH(float tempC)
{
    float v = (analogRead(PH_PIN) / 1024.0f) * 5000.0f;
    float val = ph.readPH(v, tempC);
    ph.calibration(v, tempC);
    if (!isnan(val)) return val;
    return pHValue;
}

```

```

float readDO(float tempC)
{
    uint16_t do_mv = adcToMillivolts(analogRead(DO_PIN));
    uint8_t t = (uint8_t)constrain((int)round(tempC), 0, 40);
    float val = (float)readDOmgL(do_mv, t);
    if (!isnan(val) && val >= 0.0f) return val;
    return doValue;
}

```

```

void drawLine(int y, const String &label, const String &value)
{
    screen.fillRect(0, y, 480, 22, 0x0000);
    screen.setCursor(0, y);
    screen.setTextColor(0xFFFF, 0x0000);
}

```

```
screen.setTextSize(2);  
screen.print(label);  
screen.print(": ");  
screen.println(value);  
}
```

```
void updateDisplay()  
{  
  drawLine(20, "Temp (C)", String(temperatureC, 2));  
  drawLine(50, "pH",    String(phValue, 2));  
  drawLine(80, "DO",    String(doValue, 2));  
  drawLine(110, "EC (uS)", String(ecValue, 0));  
  drawLine(140, "NH4",   String(ammoniumValue, 2));  
  drawLine(170, "NO3",   String(nitrateValue, 2));  
  drawLine(200, "Flow",   String(flowValue, 2));  
}
```

```
void tryParseUartSensors()  
{  
  while (Serial1.available())  
  {  
    char c = (char)Serial1.read();  
    if (c == '\r') continue;
```

```
if (c == '\n')
{
    rxLine.trim();
    if (rxLine.length())
    {
        if (rxLine.startsWith("NH4=")) ammoniumValue =
rxLine.substring(4).toFloat();

        else if (rxLine.startsWith("NO3=")) nitrateValue =
rxLine.substring(4).toFloat();

        else if (rxLine.startsWith("FLOW=")) flowValue =
rxLine.substring(5).toFloat();

        else
        {
            float v = rxLine.toFloat();
            if (v != 0.0f) ammoniumValue = v;
        }
    }
    rxLine = "";
}
else
{
    if (rxLine.length() < 64) rxLine += c;
}
```

```
}  
}
```

```
void sendPacketToMKR()  
{  
    String pkt;  
    pkt.reserve(96);  
    pkt += "T=" + String(temperatureC, 2);  
    pkt += ",PH=" + String(phValue, 2);  
    pkt += ",DO=" + String(doValue, 2);  
    pkt += ",EC=" + String(ecValue, 0);  
    pkt += ",NH4=" + String(ammoniumValue, 2);  
    pkt += ",NO3=" + String(nitrateValue, 2);  
    pkt += ",FLOW=" + String(flowValue, 2);  
    Serial1.println(pkt);  
    Serial.println(pkt);  
}
```

```
void setup()  
{  
    Serial.begin(115200);  
    Serial1.begin(9600);
```

```
screen.begin();  
screen.fillScreen(0x0000);
```

```
ph.begin();  
ec.setCalibration(1);
```

```
screen.setRotation(1);  
screen.setCursor(0, 0);  
screen.setTextColor(0xFFFF, 0x0000);  
screen.setTextSize(2);  
screen.println("EMWAC Sensor Node");  
}
```

```
void loop()  
{  
  tryParseUartSensors();  
  
  temperatureC = readTemperatureC();  
  phValue = readPH(temperatureC);  
  doValue = readDO(temperatureC);  
  ecValue = readECuscm(temperatureC);  
  
  updateDisplay();
```

```
sendPacketToMKR();
```

```
delay(1000);
```

```
}
```

Satellite codes

```

#include <arduino_lsm.h>
#include <arduino_lsm_cli_support.h>
#include <IWatchdog.h>
#include <time.h>
#include <array>

using namespace ArduinoLSM;

#if defined(ARDUINO_BOARD_LS200)
LS200SX1262Interface lsm;
#elif defined(ARDUINO_BOARD_NUCLEO_L476_SX1261) ||
defined(ARDUINO_BOARD_NUCLEO_WB55_SX1261)
NucleoSX1261Interface lsm;
#elif defined(ARDUINO_BOARD_NUCLEO_L476_SX1262) ||
defined(ARDUINO_BOARD_NUCLEO_WB55_SX1262)
NucleoSX1262Interface lsm;
#elif defined(ARDUINO_BOARD_NUCLEO_L476_LR1110) ||
defined(ARDUINO_BOARD_NUCLEO_WB55_LR1110)
NucleoLR1110Interface lsm;
#elif defined(ARDUINO_BOARD_NUCLEO_WL55)
NucleoWL55JC1Interface lsm;
#elif defined(ARDUINO_BOARD_LS300)
LS300Interface lsm;
#else
# error "unsupported board"
#endif

LSM::SatMAC sat_mac(lsm.radio_manager);
SerialAggregator serial;
std::array<uint8_t, 8192> workspace;
uint32_t workspace_size;

static void configure_for_variant();
static void two_second_led();
static void sync_integration_test();

extern const CLI::Command cli_commands;

extern "C" {

    void SystemClock_Config_Before(void)
    {
        // set to protect the code before setup()
        IWatchdog.begin(3000000);
    }
};

void setup()
{
    configure_for_variant();

    // change the log level here
    serial.set_log_level(LogLevel::Trace);

    // the light comes on for two seconds while the thread blocks

```

```

two_second_led();

// attach LSM to physical layer
lsm.start();

// LSM will bootstrap by default
// sat_mac.set_bootstrap_permitted(false);

// change the country code
sat_mac.set_country(LSM::CountryCode::kGBR);

// change the location
sat_mac.set_location(51.573786, -1.314538);

// change the keys
{
    static const LSM::AES::Key apps_key{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
    static const LSM::AES::Key nwks_key{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};

    (void)LSM::HAL::set_key(LSM::KeyID::SatApps0, apps_key);
    (void)LSM::HAL::set_key(LSM::KeyID::SatNwks0, nwks_key);
}

// change network settings, but only if they haven't already been set
if(!sat_mac.network_is_active()){

    LSM::SatActivationParam arg = {

        .dev_addr = 0,
        .up_counter = 0
    };

    (void)sat_mac.add_network(arg);
}

// an n second delay used by LacunaSpace to sync integration test
sync_integration_test();

// satellite protocol will start working now
sat_mac.start();
}

void loop()
{
    IWatchdog.reload();

    // make the CLI work
    serial.process();

    // make LSM work
    lsm.process();

    // times are in microseconds
    static uint64_t time_of_last_send;

```

```

    static const uint64_t send_delay = 5*60*1000000;           // send no more
frequently than this delay
    static const uint64_t long_duration_pass = 5*60*1000000;   // app
specific definition of a long duration pass

    LSM::SatMAC::UplinkInfo info;

    if(sat_mac.uplink_is_available() && sat_mac.get_uplink_info(info)){

        // We need some kind of uplink strategy in this simple application.
        //
        // For a short pass, we will send once at TCA.
        // For a long pass, we will send many times with a delay between each
send
        //
        // In your own app use any strategy you like.
        //
        if(
            (
                ((info.end - info.start) >= long_duration_pass)
                ||
                (LSM::HAL::get_real_time() >= info.tca)
            )
            &&
            (
                (time_of_last_send == 0)
                ||
                ((LPTimer::get_ticks() - time_of_last_send) > send_delay)
            )
        ){

            // change the message
            static const char msg[] = "helloworld";

            (void)sat_mac.uplink(1, msg, strlen(msg));

            time_of_last_send = LPTimer::get_ticks();
        }
    }

    IWatchdog.reload();

    Singletons::control.sleep();
}

static void sync_integration_test()
{
    serial.start_event("reset reason=");
    serial.next_part(CLICodec::encode_reset_reason(Singletons::control.get_res
et_reason()));
    serial.end();

    uint64_t wait_until = LPTimer::get_ticks() + 5000000;

    while(LPTimer::get_ticks() < wait_until){

```

```

        IWatchdog.reload();

        serial.process();

        lsm.process();

        Singletons::control.sleep();
    }
}

static void two_second_led()
{
    digitalWrite(LED_BUILTIN, HIGH);
    IWatchdog.reload();
    delay(2000);
    IWatchdog.reload();
    digitalWrite(LED_BUILTIN, LOW);
}

static void configure_for_variant()
{
    // adjust to slightly longer than the longest recurring interrupt
    IWatchdog.set(uint32_t(LPTimer::get_overflow_ticks()) +
uint32_t(1000000));

#ifdef ARDUINO_BOARD_LS300 || defined(ARDUINO_BOARD_LS200)

    #   ifndef USBD_USE_CDC
        Serial.begin(115200);
        serial.set_port(Serial, 0);
    #   else
        Serial.begin(38400);
        serial.set_port(Serial, 0);
    #   endif

    #   if defined(LS_EXT_UART)
        if(reinterpret_cast<Stream *>(&LS_EXT_UART) != reinterpret_cast<Stream
*>(&Serial)){

            LS_EXT_UART.begin(38400);
            serial.set_port(LS_EXT_UART, 1);
        }
    #   endif
#else
    Serial.begin(38400);
    serial.set_port(Serial, 0);
#endif

    serial.set_command(cli_commands);

    {
        const LSM::SatMAC::Callback cb = {
            .uplink_complete = [](void *ctx, LSM::Status status){

```

```

        (void)ctx;

        serial.start_event("service_complete");
        serial.next_token("sat_uplink");
        serial.next_token(LSM::Utility::status_to_s(status));
        serial.end();
    },
    .contact_started = [](void *ctx)
    {
        (void)ctx;

        serial.put_event("sat_contact_started");
    },
    .contact_ended = [](void *ctx)
    {
        (void)ctx;

        serial.put_event("sat_contact_ended");
    },
    .almanac_downlink_complete = [](void *ctx)
    {
        (void)ctx;

        serial.put_event("sat_almanac_downlink_complete");
    },
    .time_from_wakeup = [](void *ctx, uint64_t unix_time, uint64_t
gps_time)
    {
        LSM::SatMAC *sat_mac = (LSM::SatMAC *) ctx;

        RealTimeClock::set_time(unix_time);

        sat_mac->time_has_changed();

        char buffer[100];
        serial.start_event("sat_time_from_wakeup");

        serial.next_token("unix_time=");
        serial.next_part(CLICodec::encode_uint64(unix_time, buffer,
sizeof(buffer)));

        serial.next_token("gps_time=");
        serial.next_part(CLICodec::encode_uint64(gps_time, buffer,
sizeof(buffer)));

        serial.end();
    },
    .clear_to_send_active = [](void *ctx)
    {
        (void)ctx;

        serial.put_event("sat_clear_to_send_active");
    },
    .clear_to_send_inactive = [](void *ctx)
    {

```

```

        (void)ctx;

        serial.put_event("sat_clear_to_send_inactive");
    }
};

sat_mac.set_callback(&sat_mac, cb);
}

const CLI::CommandRecord command_table[] = {
{
    "help",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseHelp arg(server, argc, argv);
    },
    nullptr
},
{
    "get_sketch_name",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){

            server.put_ack("satellite");

        }
    },
    nullptr
},
{
    "get_version",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseGetVersion(server, argc, argv)){

            server.put_ack(LSM::get_version_string());

        }
    },
    nullptr
},
{
    "soft_reset",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseSoftReset(server, argc, argv)){

            lsm.stop();
            server.put_ack();
            server.flush();
            Singletons::control.soft_reset();

        }
    },
    nullptr
},
};

```

```

{
    "set_location",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSetLocation arg(server, argc, argv);

        if(arg){
            sat_mac.set_location(arg.lat, arg.lon);
        }
    },
    nullptr
},
{
    "set_country",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSetCountry arg(server, argc, argv);

        if(arg){
            sat_mac.set_country(arg.code);
        }
    },
    nullptr
},
{
    "get_real_time",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        char buffer[32];

        if(ParseGetRealTime(server, argc, argv)){
            server.put_ack(
                CLICodec::encode_uint64(LSM::HAL::get_real_time(), buffer,
sizeof(buffer))
            );
        }
    },
    ParseGetRealTime::get_help
},
{
    "set_real_time",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSetRealTime arg(server, argc, argv);

        if(arg){
            if(RealTimeClock::set_time(arg.unix_epoch)){
                sat_mac.time_has_changed();

                server.put_ack();
            }
        }
    }
},
{
    "help",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseHelp arg(server, argc, argv);

        if(arg){
            server.put_ack();
        }
    }
},
{
    "exit",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseExit arg(server, argc, argv);

        if(arg){
            server.put_ack();
        }
    }
}
};

int main(int argc, char **argv)
{
    CLI::App app{"Satellite MAC (sat_mac)"};
    app.add_subcommand("help", "Print this help message");
    app.add_subcommand("exit", "Exit the program");
    app.add_subcommand("set_location", "Set the location of the satellite");
    app.add_subcommand("set_country", "Set the country of the satellite");
    app.add_subcommand("get_real_time", "Get the real time of the satellite");
    app.add_subcommand("set_real_time", "Set the real time of the satellite");

    CLI::Server server(app);
    server.run(argc, argv);
}

```

```

        }
        else{
            server.put_nak("out_of_range");
        }
    }
},
ParseSetRealTime::get_help
{
    "real_time_is_valid",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseRealTimeIsValid(server, argc, argv)){
            server.put_ack(
                CLICodec::encode_bool(
                    LSM::HAL::real_time_is_valid()
                )
            );
        }
    },
    ParseRealTimeIsValid::get_help
},
{
    "set_key",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSetKey arg(server, argc, argv);

        if(arg){
            if(LSM::HAL::set_key(arg.key_id, arg.key)){
                server.put_ack();
            }
            else{
                server.put_nak("error");
            }
        }
    },
    ParseSetKey::get_help
},
{
    "get_key",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        LSM::AES::Key key;

        char buffer[100];

        ParseGetKey arg(server, argc, argv);

        if(arg){

```

```

        key.fill(0);

        if(LSM::HAL::get_key(arg.key_id, key)){
            server.put_ack(CLICodec::encode_aes_key(key, buffer,
sizeof(buffer)));
        }
        else{
            server.put_nak("error");
        }
    }
},
ParseGetKey::get_help
},
{
    "set_log_level",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSetLogLevel arg(server, argc, argv);

        if(arg){
            serial.set_log_level(arg.level);
        }
    },
    nullptr
},
{
    "get_log_level",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){
            server.put_ack(
                CLICodec::encode_log_level(
                    serial.get_log_level()
                )
            );
        }
    },
    nullptr
},
{
    "sat_add_network",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSatActivate arg(server, argc, argv);

        if(arg){
            LSM::SatActivationParam param;

            param.up_counter = arg.up_counter;

```

```

        param.dev_addr = arg.dev_addr;

        if(sat_mac.add_network(param, arg.net)){
            server.put_ack();
        }
        else{
            server.put_nak("error");
        }
    }
},
ParseSatActivate::get_help
},
{
    "sat_remove_network",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSatDeactivate arg(server, argc, argv);

        if(arg){
            sat_mac.remove_network(arg.net);
        }
    },
    ParseSatDeactivate::get_help
},
{
    "sat_remove_all_networks",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){
            sat_mac.remove_all_networks();
        }
    },
    [ ](){ return "remove all network sessions"; }
},
{
    "sat_network_is_active",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSatIsActive arg(server, argc, argv);

        if(arg){
            server.put_ack(
                CLICodec::encode_bool(
                    sat_mac.network_is_active(arg.net)
                )
            );
        }
    },
    ParseSatIsActive::get_help
},

```

```

{
    "get_factory_eui64",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){

            LSM::EUI64 value;
            char buffer[20];

            if(!FactoryEUI::get_eui64(value)){

                server.put_nak("not_available");
            }
            else{

                server.put_ack(
                    CLICodec::encode_eui(
                        value,
                        buffer,
                        sizeof(buffer)
                    )
                );
            }
        },
        nullptr
    },
    {
        "clear_all_config",
        [](CLI::Server& server, unsigned argc, const char **argv)
        {
            if(ParseNoArgument(server, argc, argv)){

                Singletons::indexed_store.reset();

                server.put_ack();
                serial.flush();

                Singletons::control.soft_reset();
            }
        },
        nullptr
    },
    {
        "sat_install_almanac",
        [](CLI::Server& server, unsigned argc, const char **argv)
        {
            ParseSatInstallAlmanac arg(server, argc, argv);

            if(arg){

                if(arg.expected_size_present && (arg.expected_size !=
workspace_size)){

                    server.put_nak("unexpected_size");
                }
            }
        }
    }
}

```

```

        }
        else if(!sat_mac.install_almanac(workspace.data(),
workspace_size)){

            server.put_nak("error");
        }
        else{

            server.put_ack();
        }
    }
},
nullptr
},
{
    "sat_clear_almanac",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){

            sat_mac.clear_almanac();
        }
    },
    nullptr
},
{
    "write_workspace",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseWriteWorkspace arg(server, argc, argv);

        if(arg){

            if(arg.offset > workspace.size()){

                server.put_nak("workspace_overrun");
            }
            else if((workspace.size() - arg.offset) < arg.size){

                server.put_nak("workspace_overrun");
            }
            else{

                workspace_size = ((arg.offset + arg.size) >
workspace_size) ? (arg.offset + arg.size) : workspace_size;
                (void)memcpy(&workspace[arg.offset], arg.chunk, arg.size);

                server.put_ack();
            }
        }
    },
    nullptr
},
{
    "clear_workspace",

```

```

[] (CLI::Server& server, unsigned argc, const char **argv)
{
    if (ParseNoArgument(server, argc, argv)) {

        workspace.fill(0);
        workspace_size = 0;

    }
},
nullptr
},
{
    "get_write_workspace_chunk_size",
[] (CLI::Server& server, unsigned argc, const char **argv)
{
    char buffer[10];

    if (ParseNoArgument(server, argc, argv)) {

        server.put_ack(CLICodec::encode_uint32(ParseWriteWorkspace::ge
t_chunk_size(), buffer, sizeof(buffer)));
    }
},
nullptr
},
{
    "get_workspace_size",
[] (CLI::Server& server, unsigned argc, const char **argv)
{
    char buffer[10];

    if (ParseNoArgument(server, argc, argv)) {

        server.put_ack(CLICodec::encode_uint32(workspace_size, buffer,
sizeof(buffer)));
    }
},
nullptr
},
{
    "get_workspace_max",
[] (CLI::Server& server, unsigned argc, const char **argv)
{
    char buffer[10];

    if (ParseNoArgument(server, argc, argv)) {

        server.put_ack(CLICodec::encode_uint32(workspace.size(),
buffer, sizeof(buffer)));
    }
},
nullptr
},
{
    "sat_set_almanac_key",
[] (CLI::Server& server, unsigned argc, const char **argv)

```

```

{
    ParseSatSetAlmanacKey arg(server, argc, argv);

    if(arg){
        if(!sat_mac.lpp.set_user_key(arg.key, sizeof(arg.key))){
            server.put_nak("error");
        }
    }
},
nullptr
},
{
    "sat_clear_almanac_key",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){
            sat_mac.lpp.clear_user_key();
        }
    },
    nullptr
},
{
    "sat_lpp_almanac_installed",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){
            server.put_ack(
                CLICodec::encode_bool(
                    sat_mac.lpp.almanac_installed()
                )
            );
        }
    },
    nullptr
},
{
    "sat_set_bootstrap_permitted",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        ParseSatSetBootstrapPermitted arg(server, argc, argv);

        if(arg){
            sat_mac.set_bootstrap_permitted(arg.value);
        }
    },
    nullptr
},
{
    "set_utc_time",
    [](CLI::Server& server, unsigned argc, const char **argv)

```

```

{
    ParseSetUTCTime arg(server, argc, argv);

    if(arg){
        RealTimeClock::set_time(arg.time);
        sat_mac.time_has_changed();
    }
},
nullptr
},
{
    "get_utc_time",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){

            char buffer[100];
            server.put_ack(
                CLICodec::encode_time(
                    RealTimeClock::get_time(),
                    buffer,
                    sizeof(buffer),
                    CLICodec::TimePrecision::Microseconds
                )
            );
        }
    },
    nullptr
},
{
    "sat_get_tx_capability",
    [](CLI::Server& server, unsigned argc, const char **argv)
    {
        if(ParseNoArgument(server, argc, argv)){

            LSM::RadioAdapterHardwareCapability cap;

            sat_mac.get_tx_capability(cap);

            server.start_ack("800=");
            server.next_part(CLICodec::encode_bool(cap.k800));

            server.next_token("900=");
            server.next_part(CLICodec::encode_bool(cap.k900));

            server.next_token("2000=");
            server.next_part(CLICodec::encode_bool(cap.k2000));

            server.end();
        }
    },
    nullptr
}

```

```

    },
    {
        "sat_get_rx_capability",
        [](CLI::Server& server, unsigned argc, const char **argv)
        {
            if(ParseNoArgument(server, argc, argv)){

                LSM::RadioAdapterHardwareCapability cap;

                sat_mac.get_rx_capability(cap);

                server.start_ack("800=");
                server.next_part(CLICodec::encode_bool(cap.k800));

                server.next_token("900=");
                server.next_part(CLICodec::encode_bool(cap.k900));

                server.next_token("2000=");
                server.next_part(CLICodec::encode_bool(cap.k2000));

                server.end();

            }
        },
        nullptr
    }
};

const CLI::Command cli_commands(command_table,
sizeof(command_table)/sizeof(*command_table));

```